Parallel Shark Smell Optimization

Pietro Cipriani
258770
pietro.cipriani@studenti.unitn.it

Filippo De Grandi
257824
filippo.degrandi@studenti.unitn.it

Giacomo Vettore
268198
giacomo.vettore@studenti.unitn.it

***Abstract*—The Shark Smell Optimization (SSO) algorithm is a metaheuristic optimization technique inspired by the hunting behavior of sharks. Due to the high computational cost of population-based optimization, this paper presents various parallel implementations of SSO using MPI, OpenMP and hybrid MPI+OpenMP programming models. The MPI version distributes the population across processes, while the hybrid approach combines distributed-memory and shared-memory parallelism to improve resource utilization on multicore systems. Performance evaluation on standard benchmark functions analyzes execution time, speedup, and scalability for different population sizes and processor configurations.**

Three decomposition strategies are investigated: shark-level (population), dimension-level, and rotation-level parallelism. Performance evaluation on the Rastrigin benchmark function with a standard configuration (np=1000, nd=200, k_max=1000, m=50) shows that shark-level decomposition achieves near-linear speedup in both MPI and OpenMP, reaching 43× with 64 MPI processes and 50× with 64 OpenMP threads. Dimension-level and rotation-level strategies exhibit poor scaling because their work granularity is too small to amortize synchronization and communication costs. The hybrid MPI+OpenMP shark-level implementation achieves the shortest wall-clock time, reaching 450× over serial at p=64, t=32 (2048 total cores, 22% efficiency). Sizing the population to the machine (np=8192) removes this starvation: the hybrid then reaches about 570× at 1024 cores at roughly 56% efficiency, and shark-level decomposition is shown to be weakly scalable as well.

***Index Terms*—parallelization, hpc, mpi, openmp, sharks, smelling**

I. Introduction

Meta-heuristic optimization algorithms have become a widespread tool for solving complex nonlinear optimization problems where deterministic or gradient-based approaches are computationally infeasible[1]. Population-based optimization methods are effective in exploring high-dimensional search spaces and handling multimodal objective functions. These techniques are widely applied in engineering optimization, machine learning, scheduling, computational biology, and scientific computing.

Although the Shark Smell Optimization (SSO) algorithm demonstrates strong optimization capabilities on nonlinear benchmark functions, its iterative population-based structure introduces substantial computational costs as the number of sharks, problem dimensionality, and iteration count increase. Parallelization becomes essential to reduce execution time and improve scalability.

This analysis studies various portions of the serial algorithm, parallelizing them in isolation, so that the observed behavior can be attributed to that specific computational step rather than to a combination of optimizations applied at once. This is the reason for the large number of variants reported here, through which we are able to identify the configuration leading to the best performance.

- `shark-level` parallelism, where candidate solutions are distributed across population elements;
- `dimension-level` parallelism, where computations across search-space dimensions are parallelized;
- `rotation-level` parallelism, where the local search around each candidate is parallelized.

These three decomposition strategies are orthogonal to the programming model: each is implemented under MPI, OpenMP, and hybrid MPI+OpenMP. "Hybrid" here denotes the programming model (distributed plus shared memory), not a fourth decomposition.

By combining these algorithmic decomposition strategies with MPI, OpenMP, and hybrid MPI+OpenMP execution models, the work evaluates multiple parallel variants of SSO and analyzes their scalability, synchronization behavior, communication overhead, and computational efficiency on multicore HPC architectures.

The study is structured as a controlled, point-by-point investigation: each independent loop of the serial algorithm is parallelized in isolation, so that the observed scaling behavior can be attributed to that specific computational step rather than to a combination of optimizations applied at once. This is what motivates the comparatively large number of variants reported here, and it is precisely this systematic exploration of each parallelizable point that lets us identify the configuration that delivers the best performance.

A. The Shark Smell Optimization Algorithm

The SSO [2] algorithm is a population-based metaheuristic inspired by sharks' ability to locate prey by sensing and following odor gradients.

The method maintains a population of candidate solutions (`sharks`) that move through the search space combining global exploration and focused local exploitation.

[1]We found this "bestiary" with a particular hot-take on meta-heuristic algorithms [1]

```c
struct Shark {
  double *position; // Current position of the shark.
  double *speed;    // Current speed of the shark.
  double pos_score; // The OF(position) value.
};
```

Listing 1. Shark data structure

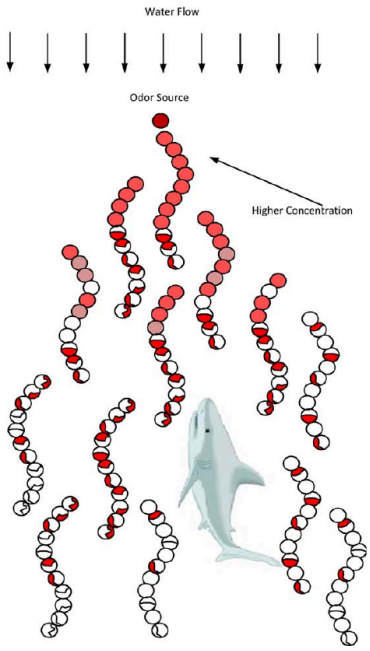


Fig. 1. Schematic illustration of a shark's movement to odor source.

Each shark iteratively updates its position and speed based on a combination of inertia (momentum from previous movement) and attraction toward promising regions of the search space (global best or local neighborhood best). Additionally, a local rotational search is performed around improved sharks to refine their positions.

The main stages of the SSO algorithm are described in the following pseudo-code, which highlights the key computational steps:

Serial Algorithm

```
// Shark loop
for shark_index in 0 .. cfg.np-1:
  shark = sharks[shark_index]
  // Stages loop
  for k in 0 .. cfg.k_max-1:
    R1 = uniform(0,1)
    R2 = uniform(0,1)

    // 1) per-dimension derivative and speed update
    for dim in 0 .. cfg.nd-1:
      derivative = eval_derivative(shark.position, cfg.nd, cfg.obj, dim)
      grad_term = cfg.eta * R1 * derivative
      mom_term  = cfg.alpha * R2 * shark.speed[dim]
      shark.speed[dim] = grad_term + mom_term
      // apply velocity limiter using cfg.beta
      if abs(shark.speed[dim]) > abs(cfg.beta * shark.speed[dim]):
        shark.speed[dim] = cfg.beta * shark.speed[dim]

    // 2) forward movement and clamp
    for dim in 0 .. cfg.nd-1:
      shark.position[dim] = clamp(shark.position[dim] + shark.speed[dim], domain[dim])

    // 3) rotational local search
    best = OF(shark.position, cfg.nd, cfg.obj)
    best_r3 = 0
    for m in 0 .. cfg.rotations-1:
      r3 = uniform(-1,1)
      for dim in 0 .. cfg.nd-1:
        candidate[dim] = clamp(shark.position[dim] * (1 + r3), domain[dim])
      val = OF(candidate, cfg.nd, cfg.obj)
      if val > best: best = val; best_r3 = r3
    if best_r3 =/= 0:
      for dim in 0 .. cfg.nd-1:
        shark.position[dim] = clamp(shark.position[dim] * (1 + best_r3), domain[dim])

    // 4) update global best
    cur_min = -best
    if cur_min < best_min:
      best_min = cur_min
      copy(best_pos, shark.position)

// result in best_min, best_pos
```

a) *Design intuition:*

- Inertia preserves momentum and enables smooth exploration of the search space.
- Attraction terms bias motion toward high-quality regions (global best or local neighborhood best).
- Rotational local search acts as an exploitation operator similar to a focused local search or mutation that samples points in a small region around a candidate to refine its position.

II. Parallel Design

The SSO algorithm contains various sources of parallelism. The evaluation and update of individual sharks can generally be executed independently, making shark-level decomposition highly suitable for distributed-memory execution. Conversely, dimension-level decomposition is a finer-grained form of parallelism that can be applied within the evaluation of a single shark's position and speed updates, which may be more efficient in shared-memory environments.

Fig. 2 shows the execution time of the serial algorithm as each parameter (number of sharks, dimensions, rotations, stages) increases. The cost grows with every parameter, so larger problem instances quickly become intractable serially and motivate the parallel designs that follow.

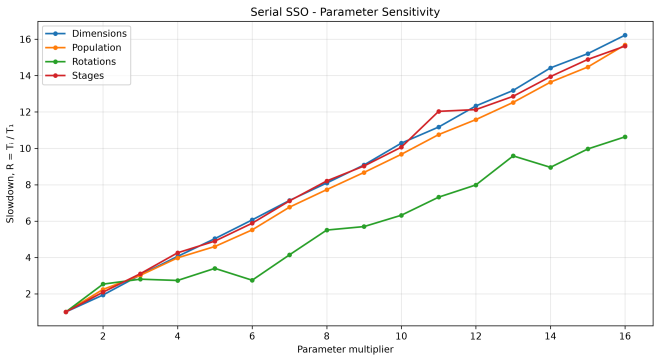


Fig. 2. Execution time of the serial SSO algorithm as a function of the number of sharks, dimensions, stages and rotations.

Although the "stage" loop over the stage index `k`, in the graph indicated by the red line, might at first seem a good candidate for parallelization, it is inherently sequential: each iteration at stage `k` consumes and modifies data produced at stage `k-1` (for example updated shark positions, velocities and fitness values). These true data dependencies prevent concurrent execution of different `k` iterations without violating correctness. Any approach to overlap would require complex checkpointing or synchronization that typically outweighs potential gains. Therefore, parallelism is applied at the shark, dimension and rotation levels rather than across stage iterations.

Each decomposition strategy we implement targets exactly one of the independent loops of the serial algorithm shown in the previous section. The outer shark loop defines `shark-level` parallelism; steps 1 and 2 (the per-dimension derivative, speed update and forward movement) define `dimension-level` parallelism; step 3 (the rotational local search over the `M` probes) defines `rotation-level` parallelism. Step 4 is the only point requiring coordination, since updating the global best is a reduction across the otherwise independent work. Parallelizing one loop at a time and benchmarking it on its own lets us measure the scaling of each step separately, rather than mixing several changes in one solver. Running each decomposition under both programming models then tells the cost of the decomposition apart from the cost of the memory model.

This work therefore evaluates various combination of the three programming models (MPI, OpenMP, and hybrid MPI+OpenMP) with the decomposition strategies above, isolating how each choice affects scalability and synchronization cost.

A. Shark-level Parallelism

The shark population is the most natural decomposition unit in SSO. Each shark evolves independently across all stages; no inter-shark communication occurs during the update loop. Assigning disjoint subsets of sharks to different workers therefore yields a straightforward parallel workload. The only required global coordination is a single reduction at the end of the run to identify the globally best solution. The communication-to-computation ratio decreases proportionally as the population grows, making shark-level decomposition strongly scalable.

A practical design consideration is PRNG independence. Each shark requires its own random stream for velocity updates (R1, R2) and rotational probes (R3). In the MPI implementation, ranks use rank-offset seeds; in the OpenMP implementation, each thread carries a thread-local seed. This avoids both contention on a shared RNG and correlation artifacts between workers.

The principal limitation of this strategy is granularity: if the population is small relative to the number of workers (e.g., np $= 1000$ with 1024 processes), some workers receive zero or one shark, wasting resources.

B. Dimension-level Parallelism

Within the velocity and position update of a single shark (Eq. 7–9 of [2]), computations across different dimensions are independent: the partial derivative of the objective function in dimension j depends only on the current position vector, not on other dimensions' derivatives. This permits fine-grained `nd`-way parallelism inside the inner loops of the SSO update.

However, several structural constraints limit the practical effectiveness of this approach:

- **Granularity**: the maximum useful worker count equals `nd` (200 in the standard configuration), and each work unit is a single gradient evaluation, which is a very lightweight operation.
- **Synchronization frequency**: before the rotational search can start, all dimension updates must complete. This creates $O(k\_\max \times n_p)$ synchronization barriers per run, which amounts to $O(10^6)$ barriers with the standard parameters.
- **Communication volume** (MPI): the complete position vector must be reconstructed on every rank after each speed update via `MPI_Allgatherv`, incurring $O(n_p \times n_d)$ bytes of communication per stage.

Dimension-level decomposition is beneficial only when the objective function evaluation is expensive and `nd` is large enough to amortize the per-barrier overhead.

C. Rotation-level Parallelism

The M rotational probes in the local search (Eq. 10 in [2]) are mutually independent: each probe draws a scalar r3 uniformly at random and evaluates the objective at a scaled position around the current shark. The best probe is then selected with a simple maximum reduction over M values.

Like shark-level decomposition, this loop is highly parallel. Unlike it, the available parallelism is capped by M rather than by the population: with M $= 50$ and 64 workers, each worker receives fewer than one probe on average, making the parallel overhead dominate. Rotation-level parallelism therefore becomes effective only when M is substantially larger than the number of workers, which corresponds to configurations where a fine-grained local search is required for solution quality. The favorable rotation configuration (M $= 20000$), analyzed in the performance section, exposes the scaling behavior of this strategy under such conditions.

D. Data Dependencies

To decide which loops can be parallelized, and with which synchronization, we classified the data dependencies of every shared variable in the serial reference implementation (`src/main_serial_v2.c`). Table I lists the representative dependencies; line numbers refer to that file, and the iteration index k denotes the stage loop while m denotes the inner rotation loop. Each entry reports the earlier and later statement (as `line, iter, access`, with access $\in \{R, W, \mathrm{RW}\}$), whether the dependency is loop-carried, and the kind of dataflow (flow, anti, or output).

TABLE I
DATA DEPENDENCIES IN THE SERIAL SSO KERNEL. INDEX k IS THE STAGE LOOP, m THE ROTATION LOOP.

Variable	Earlier	Later	Carried?	Kind
`speed[dim]`	81, k, W	73, $k+1$, R	yes (k)	flow
`speed[dim]`	73, k, R	81, k, W	no	anti
`position[dim]`	97, k, W	75, $k+1$, R	yes (k)	flow
`position[dim]`	75, k, R	97, k, W	no	anti
`best`	121, m, W	120, $m+1$, R	yes (m)	flow
`best_r3`	122, m, W	122, $m+1$, W	yes (m)	output
`scratch[dim]`	113, m, W	119, m, R	no	flow
`scratch[dim]`	113, m, W	113, $m+1$, W	yes (m)	output
`best_min`	137, k, W	136, $k+1$, R	yes (k)	flow
`_seed` (PRNG)	69, k, RW	110, k, RW	yes	flow

The resolution of each dependency in the parallel variants is summarized in Table II. Most are removed by privatization (per-worker scratch buffers and independent PRNG streams) or by a reduction; the global best is folded with a critical section. Two cases deserve a closer look and are discussed below.

TABLE II
RESOLUTION OF EACH DATA DEPENDENCY IN THE OPENMP VARIANTS.

Variable	Dependency	Resolution (OpenMP)
loop counters	none	partition with `#pragma omp for`
`_seed` (R1, R2, r3)	flow (RNG stream)	independent per-worker streams (`rand_r` + thread seed). Actually the flow is modified, however the flow of a RNG is not sufficiently relevant to justify further synchronization
`scratch` / candidate	flow + output (m)	privatize (per-thread buffer). The flow dependency is not parallelized in any variant
`best`, `best_r3`	flow + output (m)	reduction (`declare reduction rot_best`)
`best_min`, `best_pos`	output + flow (k, sharks)	privatize + `#pragma omp critical`
`position`, `speed` (intra-stage)	anti	loop fission: two `#pragma omp for` with the implicit barrier between them
`position`, `speed` (inter-stage)	flow (k)	none: the stage loop stays serial

Critical dependency: the stage loop. The only dependency with no parallel resolution is the flow dependency carried across the stage loop: `position`, `speed`, and `best_min` written at stage k are read at stage $k+1$. Unlike the others, this cannot be privatized or reduced away, because stage $k+1$ genuinely consumes the state produced at stage k. This is the formal reason the k loop is executed serially in every variant, and why parallelism is applied at the shark, dimension and rotation levels instead. It contrasts sharply with `best` and `best_r3`, which are also loop-carried but only across the rotation loop m, where the carry is an associative maximum and therefore collapses cleanly into a reduction.

Critical dependency: the dimension loops. Within a single stage, the speed update (line 81) and the forward-movement update (line 97) cannot be fused into one parallel loop over dimensions. The derivative read at line 75 may depend on the entire position vector (for example the Griewangk product term), so every per-dimension speed update must complete before any position component is overwritten. The two loops are therefore kept separate, relying on the implicit barrier at the end of the first `#pragma omp for`; fusing them would introduce a read-after-write race on `position`.

III. IMPLEMENTATION

All code and scripts related to the project are available in the repository [3]. Shared utilities and data structures live in `sso/sso.c`, while multiple execution strategies are exposed through self-contained entrypoints in the `src` direc-

tory, e.g. `src/main_mpi_sharks.c`, `src/main_openmp_dim.c`, `src/main_hybrid_dim.c`, etc. Each entrypoint implements a specific parallelization strategy and programming model on top of the same SSO formulation.

The names of the entrypoints follow the convention `main_<model>_<strategy>.c`, where `<model>` is one of `mpi`, `openmp`, or `hybrid`, and `<strategy>` indicates the parallelization approach, i.e. `sharks`, `dim`, `rot`.

The following subsections describe the main parallelization strategies we implemented, the key code locations, and the design trade-offs.

A. MPI Implementations

We implemented three MPI decompositions that map naturally to the SSO algorithm:

- **Shark-level (population) decomposition**: each MPI rank owns a subset of the population and executes the full per-shark stages locally. At the end of the run ranks perform a reduction to determine the globally best solution. See `src/main_mpi_sharks.c`. This approach is straightforward and minimizes communication during the main loop, as each rank can independently update its sharks. This results in low communication overhead and good scalability when the population size is large enough to amortize the cost of setup and evaluation. However, it may suffer from load imbalance if the sharks are not uniformly distributed in terms of computational cost (e.g. some sharks may converge faster than others).
- **Dimension-level decomposition**: each shark's position vector is partitioned across MPI ranks; each rank computes updates for its set of dimensions and `MPI_Allgatherv` is used to reconstruct full position vectors when needed. See `src/main_mpi_dim.c`. Here, the main loop is parallelized across dimensions, which can be beneficial when the cost of evaluating the objective function or computing gradients is dominated by per-dimension work (e.g. very high `nd`). However, this approach introduces higher communication overhead due to the need for frequent all-gather operations to share updated positions across ranks, and it requires more complex synchronization to ensure consistency of the shared state.

```c
// After updating its slice [start_dim, end_dim), every rank must rebuild
// the full position vector. Issued once per shark per stage (~10^6 times).
MPI_Allgatherv(shark_ptr->position + start_dim, end_dim - start_dim, MPI_DOUBLE,
               shark_ptr->position, scatter_sizes, scatter_starts,
               MPI_DOUBLE, MPI_COMM_WORLD);
```

Listing 2. Dimension-level MPI: the per-stage all-gather that reassembles the position vector. Its frequency is the main scaling bottleneck (Section 4).

- **Rotation-level decomposition**: the inner rotational local-search probes are distributed across ranks and the best candidate is found with a reduction (e.g. `MPI_MAXLOC` or custom two-value reduction). See `src/main_mpi_rot.c`. Like the dimension-level approach, this strategy can be effective when the cost of the rotational search is significant (e.g. large `rotations`), but it also introduces communication overhead due to the need for reductions to find the best candidate across ranks. However, it minimizes redundant objective evaluations since each rank only evaluates a subset of the rotations, and the reduction combines results efficiently.

```c
// Each rank scans its probe slice into buf, then a single MAXLOC reduction
// keeps the best (value, r3) pair across all ranks in one collective.
struct { double best, best_r3; } buf = { OF(shark_ptr->position, cfg.nd, cfg.obj), 0.0 };
for (uint32_t m = start_rot; m < end_rot; ++m) { /* probe, update buf */ }
MPI_Allreduce(MPI_IN_PLACE, &buf, 1, MPI_2DOUBLE_PRECISION, MPI_MAXLOC, MPI_COMM_WORLD);
```

Listing 3. Rotation-level MPI: a two-value `MAXLOC` reduction selects the winning probe and its `r3` across ranks.

B. OpenMP Implementations

OpenMP variants exploit shared memory to avoid message passing; the code, like the MPI variants, uses different decomposition strategies depending on which loop is most expensive.

- **Shark-level (thread over sharks)**: each OpenMP thread processes different sharks end-to-end and maintains thread-local scratch buffers and per-thread best values to reduce contention. See `src/main_openmp_sharks.c`. A parallel region is created through `#pragma omp parallel` and the outer loop over sharks is partitioned with `#pragma omp for`. Each thread maintains its own RNG state and scratch arrays to avoid data races and false sharing. At the end of the loop, a reduction is performed to find the global best solution across threads.
- **Dimension-level (thread over dimensions)**: A single `#pragma omp parallel` region is opened outside all loops; the thread team then processes the outer shark and stage loops collaboratively. Within each stage, the velocity update and position update loops are parallelized with `#pragma omp for schedule(static)`, while the random scalars R1 and R2 are drawn inside a `#pragma omp single` block to guarantee all threads use a consistent pair. The rotational search is also guarded by `#pragma omp single`, so it runs on one thread while the others wait at the implicit barrier. See `src/main_openmp_dim.c`.
- **Rotation-level (parallel rotation probes)**: A single `#pragma omp parallel` region is opened once, outside the shark and stage loops; the thread team then walks those loops together. Within each stage the serial work (the velocity and position update) runs inside a `#pragma omp single` block, while the inner rotation loop is shared with `#pragma omp for schedule(static)`. A custom OpenMP reduction over a `RotationBest` struct (declared with `#pragma omp declare reduction`) finds the best (r3, value) pair across threads without a critical section. Each thread holds its own candidate buffer and per-thread seed (derived from `seed_base + (tid << 16)`) to avoid

false sharing and PRNG contention. Opening the region once avoids per-stage thread fork/join, but two implicit barriers per stage (the `single` and the `for`) remain. See `src/main_openmp_rot.c`.

```c
// Custom reduction: keep the probe with the highest objective value, along
// with its r3, so the rotation loop needs no critical section.
#pragma omp declare reduction(rot_best : struct RotationBest : \
    omp_out = (omp_in.best > omp_out.best ? omp_in : omp_out)) \
    initializer(omp_priv = (struct RotationBest){ -INFINITY, 0.0 })

#pragma omp for schedule(static) reduction(rot_best: rot_best)
for (uint32_t m = 0; m < cfg.rotations; ++m) { /* probe → rot_best */ }
```

Listing 4. Rotation-level OpenMP: a user-defined reduction over the `(value, r3)` pair replaces an explicit critical section.

OpenMP versions emphasize minimizing synchronization points and using per-thread storage for temporary arrays.

C. Hybrid MPI+OpenMP Implementations

The hybrid implementation (`src/main_hybrid_sharks.c`) combines MPI shark-level decomposition with intra-rank OpenMP parallelism, applying shark-level decomposition on both levels. Each MPI rank owns a subset of the population, and within a rank the OpenMP team splits that local block again with `#pragma omp for`, so each thread evolves its own sharks through all stages end-to-end. This two-level hierarchy maps naturally to multi-node, multi-core clusters: MPI handles coarse-grained distribution across nodes, while OpenMP exploits the shared memory within each node without message-passing overhead. Because both levels decompose the population, the only coordination is folding per-thread bests with a critical section, then a single inter-rank reduction.

```c
#pragma omp parallel num_threads(thread_num)
{
    unsigned seed = seed_base + tid;      // seed_base already carries + rank
    #pragma omp for schedule(static)      // split the rank's local sharks
    for (size_t shark = rank; shark < cfg.np; shark += size) { /* stages */ }
}
// one collective folds per-rank bests into the global best
MPI_Allreduce(&best_min, &global_best, 1, MPI_DOUBLE, MPI_MIN, MPI_COMM_WORLD);
```

Listing 5. Hybrid MPI+OpenMP: MPI distributes the population across ranks, OpenMP splits each rank's share across threads, and a single reduction combines the results.

Key design choices in the hybrid implementation:

- Each rank seeds its local PRNG with `seed_base + rank` to ensure statistically independent random streams across nodes.
- Each OpenMP thread uses a further per-thread seed `seed_base + rank + (tid << 16)` to avoid intra-rank correlations.
- A single `MPI_Allreduce` with a custom two-value struct reduction at the end combines local best values into the global best, minimizing synchronization to one collective call per run.
- The PBS script (`tests/sharks/hybrid_sharks`) uses `--map-by ppr:N:node:PE=T` with `OMP_PLACES=cores` and `OMP_PROC_BIND=close` to pin OpenMP threads to adjacent cores.

The total worker count is $P \times T$ where P is the number of MPI processes and T is the number of OpenMP threads per process. For a fixed total of $P \times T$ workers, the hybrid configuration typically outperforms the pure-MPI or pure-OpenMP equivalents by reducing the number of expensive `MPI_Allreduce` calls (fewer ranks) while still exploiting all available cores.

D. Notes and Supporting Scripts

Shared utilities for population allocation, domain setup, and initialization live in `src/sso/sso.c` and are used by every entrypoint, keeping data layout and initialization identical across variants.
Argument parsing and default parameters live in `src/sso/parse_args.c` and are extended in each `main_*` file to add parallel-specific options (only number of threads in this case). Benchmarking and plotting tools are under `benchmarks/` and `tests/` (examples: `tests/launch_tests.sh`, `benchmarks/plot_results.py`). These scripts were used to generate the figures reported in Section 4.

IV. Performance and Scalability Analysis

In this section, we present a performance analysis of the various parallel implementations of the SSO algorithm. We evaluate execution time, speedup and efficiency across the different parallelization strategies and programming models. The experiments are conducted on a multicore HPC cluster, and we analyze both strong and weak scalability. The section is organized as follows:

- Experimental setup and the fixed and favorable parameter families
- Fixed configuration: varying processes, threads, and hybrid
- Favorable shark configuration
- Weak scaling
- Analysis with different execution modes in the PBS system.

A. Experimental Setup and Benchmark Functions

Experiments use two parameter families. The first is a **fixed** (common) configuration, identical for every strategy, so the overlaid MPI/OpenMP/hybrid results are an apples-to-apples comparison:

- Number of sharks (`np`): 1000
- Number of dimensions (`nd`): 200
- Number of stages (`k_max`): 1000
- Number of rotations (`rotations`): 50

The second is a set of **favorable**, strategy-specific configurations. Each one enlarges the parallel axis of its strategy (`np` for shark-level, `nd` for dimension-level, `M` for rotation-level) well beyond the maximum worker count, and keeps enough work per unit to amortize synchronization and communication. The fixed configuration is deliberately neutral, but at high worker counts it starves the parallel axis: with `np` $= 1000$ and a 2048-core hybrid run, each core owns fewer than one shark, which depresses efficiency for reasons unrelated to the algorithm. The favorable sets remove that artifact.

TABLE III
FIXED AND FAVORABLE PARAMETER FAMILIES. EACH FAVORABLE SET HAS ITS OWN SERIAL BASELINE $T(1)$.

Configuration	np	nd	k_max	M
Fixed (common)	1000	200	1000	50
Favorable sharks	8192	100	250	50
Favorable dim	5	10^8	5	0
Favorable rot	16	200	100	20000

Each favorable configuration is timed against its own serial baseline $T(1)$, so the reported speedup and efficiency are always relative to the matching reference. The remaining algorithm parameters (`eta`, `alpha`, `beta`) are set to the values from the original SSO paper [2].

A set of scripts was developed to automate job submission across different process and thread counts. The scripts generate PBS job files from templates and throttle submission to stay within the queue's concurrent-job limit. All jobs run in `excl` placement mode to prevent resource sharing with other workloads and ensure consistent wall-clock measurements.

```shell
...
for procs in "${PROC_VALUES[@]}"; do
  for thrds in "${THRD_VALUES[@]}"; do
    while [[ $(qstat -u $USER | wc -l) -ge 30 ]]; do
      sleep 10
    done
    sed \
      -e "s/\${N_THRD}/$thrds/g" \
      -e "s/\${N_PROC}/$procs/g" \
      -e "s/\${JOB_NAME}/$JOB_NAME/g" \
      -e "s/\${N_CPUS}/$N_CPUS/g" \
      -e "s/\${PLACE}/$PLACE/g" \
      -e "s/\${P}/$P/g" \
      -e "s/\${D}/$D/g" \
      -e "s/\${K}/$K/g" \
      -e "s/\${M}/$M/g" \
      "$EXEC" | qsub
  done
done
```

Listing 6. A simplified portion of the script used to launch the tests on the HPC cluster. The script iterates over different values of processes and threads (user-defined), checks for the number of running jobs, and submits new jobs using `qsub` with the appropriate parameters.

```shell
#!/bin/env bash
#PBS -l select=${N_CPUS}:ncpus=${N_PROC}:mpiprocs=${N_PROC}:mem
#PBS -l place=${PLACE}:excl
#PBS -l walltime=0:5:00
#PBS -q shortCPUQ

# output & error files
#PBS -N ${JOB_NAME}_${N_PROC}_${N_THRD}

module load OpenMPI/4.1.6-GCC-13.2.0

mpirun -n $(( ${N_CPUS} * ${N_PROC} )) ./parallel-sso/sso_hybrid_sharks -p ${P} -d ${D} -k ${K} -m ${M} -t ${N_THRD}
```

Listing 7. An example PBS script, edited by Listing 6. The script specifies resource requirements, loads necessary modules, and runs the executable with the defined parameters for sharks, dimensions, stages, rotations, and threads.

The implementation contains three different objective functions (in `sso/ofuncs.c`), taken from the original SSO paper, that are used for testing the algorithm's performance and scalability. In our experiments, we used the Rastrigin function [4] (set as default in the code), as the differences between the various objective functions were negligible in terms of execution time and speedup.

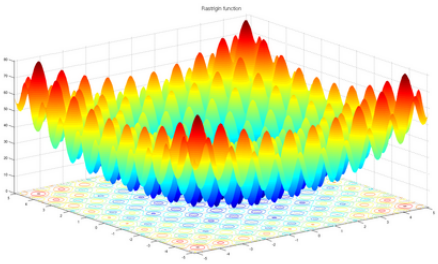


Fig. 3. Rastrigin function, a common benchmark for optimization algorithms.

The next three subsections (varying processes, varying threads, and hybrid) all use the **fixed** configuration of Table III, so the strategies and programming models are compared under identical conditions. The favorable configurations and weak scaling follow afterwards.

B. Varying number of processes (fixed configuration)

Fig. 4, Fig. 5, and Fig. 6 show execution time for the three MPI decomposition strategies with 1 to 64 processes. The OpenMP results appear in the same plots to allow direct comparison; their analysis follows in the next subsection.

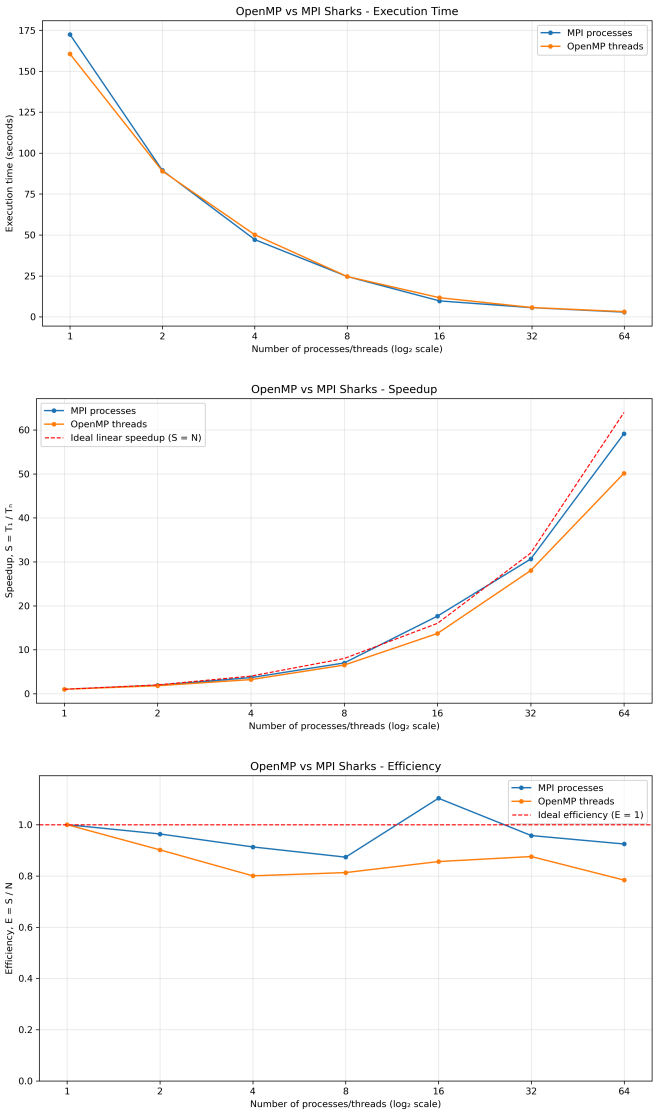


Fig. 4. Shark-level decomposition: execution time, speedup, and efficiency for MPI (processes) and OpenMP (threads).

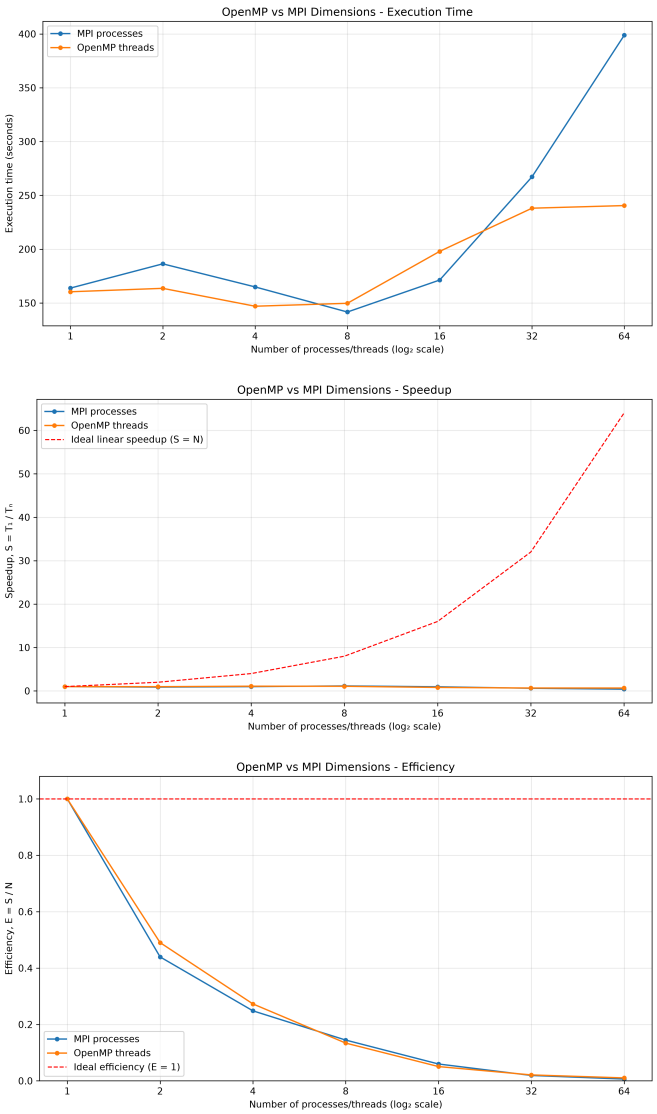


Fig. 5. Dimension-level decomposition: execution time, speedup, and efficiency for MPI (processes) and OpenMP (threads).

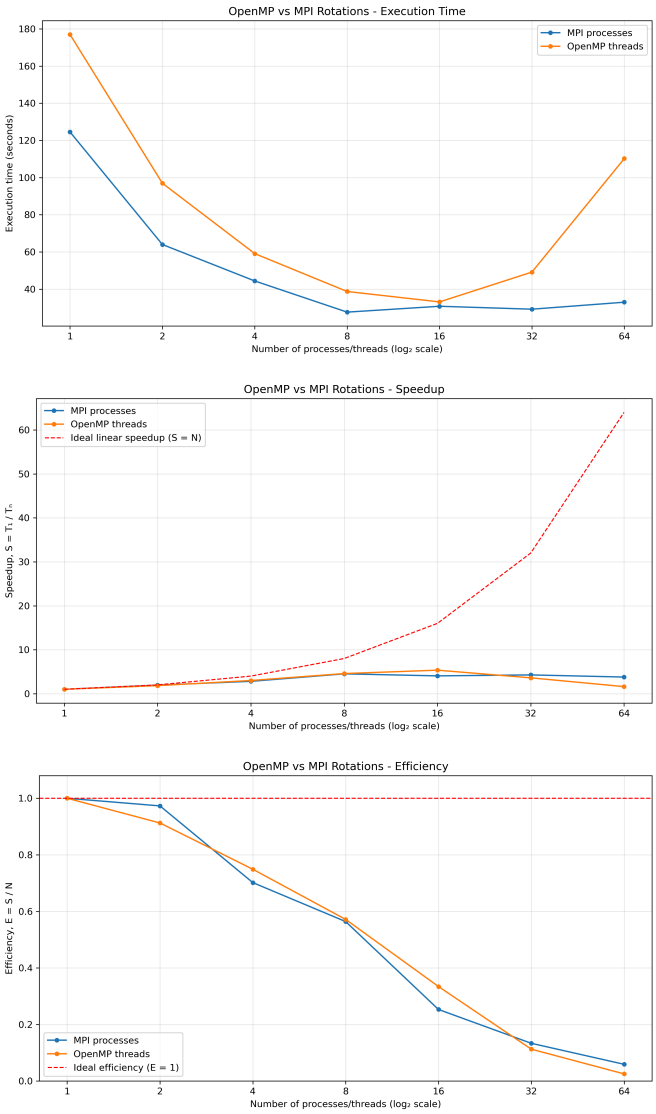


Fig. 6. Rotation-level decomposition: execution time, speedup, and efficiency for MPI (processes) and OpenMP (threads). Standard configuration: m=50.

Shark-level MPI. The single-process baseline is approximately 124 s. Execution time halves at every doubling up to 16 processes (speedup 12.7×) and continues to improve to 2.9 s at 64 processes (speedup 42.7×, efficiency 67%). The mild super-linear behaviour visible at 8–16 processes is attributable to each rank's local population fitting more readily into cache. The efficiency decline at 64 processes reflects the growing relative cost of the final `MPI_Allreduce`.

Dimension-level MPI. This variant shows no useful speedup. The single-process time of approximately 164 s rises to 399 s at 64 processes, which is more than 2.4× slower. The root cause is the `MPI_Allgatherv` call required to reassemble the full position vector after each speed update: with np=1000 sharks and k_max=1000 stages the run issues roughly 10^6 all-gather calls, each transferring 200 doubles. At high process counts the all-gather latency and bandwidth demand dominate the computation entirely.

Rotation-level MPI. With only 50 rotations per shark per stage, speedup saturates quickly. At 2 processes the speedup is 1.95× (near-ideal); at 8 processes it reaches 4.5× (efficiency 56%); beyond 8 processes performance degrades because each rank evaluates fewer than 7 probes per stage and the synchronization cost of the two-value `MPI_Allreduce` outweighs the saved computation. The favorable rotation configuration later in this section shows that this is a granularity limit, not an algorithmic one.

C. *Varying number of threads (fixed configuration)*

The OpenMP results are included in the same figures as their MPI counterparts (Fig. 4, Fig. 5, Fig. 6) to allow direct comparison.

Shark-level OpenMP. Results closely follow the MPI shark variant. The single-thread baseline is approximately 160 s (slightly higher than the MPI p=1 baseline because OpenMP thread-team initialisation is charged to the timed region). At 64 threads the time drops to 3.2 s, giving a speedup of 50× and efficiency of 78%. The shared-memory model eliminates all inter-process communication; the only synchronization is a single `#pragma omp critical` block at the end of the parallel region to fold per-thread best values into the global best. OpenMP therefore achieves higher efficiency than MPI at the same worker count, as the communication term is entirely absent.

Dimension-level OpenMP. Performance is essentially flat or worse across all thread counts. The single-thread time is approximately 160 s; at 64 threads it reaches approximately 241 s, a slowdown of 1.5×. Each `#pragma omp for` loop carries an implicit barrier at its end, generating one barrier per dimension-update pass, per shark, per stage. With the standard parameters this amounts to $2 \times n_p \times k_{\max} = 2 \times 10^6$ barriers per run, each with negligibly small work inside. The synchronization overhead dominates entirely, confirming that dimension-level parallelism requires a much heavier per-dimension kernel to be viable.

Rotation-level OpenMP. The standard (m=50) variant scales poorly. The parallel region is opened only once, so there is no per-stage thread fork/join, but each stage still pays two implicit barriers (the `#pragma omp single` that runs the serial velocity and position update, and the `#pragma omp for` over the probes), for $2 \times n_p \times k_{\max} = 2 \times 10^6$ barriers per run. With only 50 probes to share, the work between those barriers is tiny, and the serial `single` section is a fixed per-stage cost. Adding threads beyond a handful therefore yields little, and the per-stage overhead keeps efficiency low. The favorable rotation configuration later in this section shows the picture improving once M is large.

D. *Hybrid parallelism (fixed configuration)*

Fig. 7 shows execution time, speedup and efficiency for the hybrid MPI+OpenMP shark-level implementation as a function of the number of threads per process, with the number of MPI processes varied from 1 to 64.

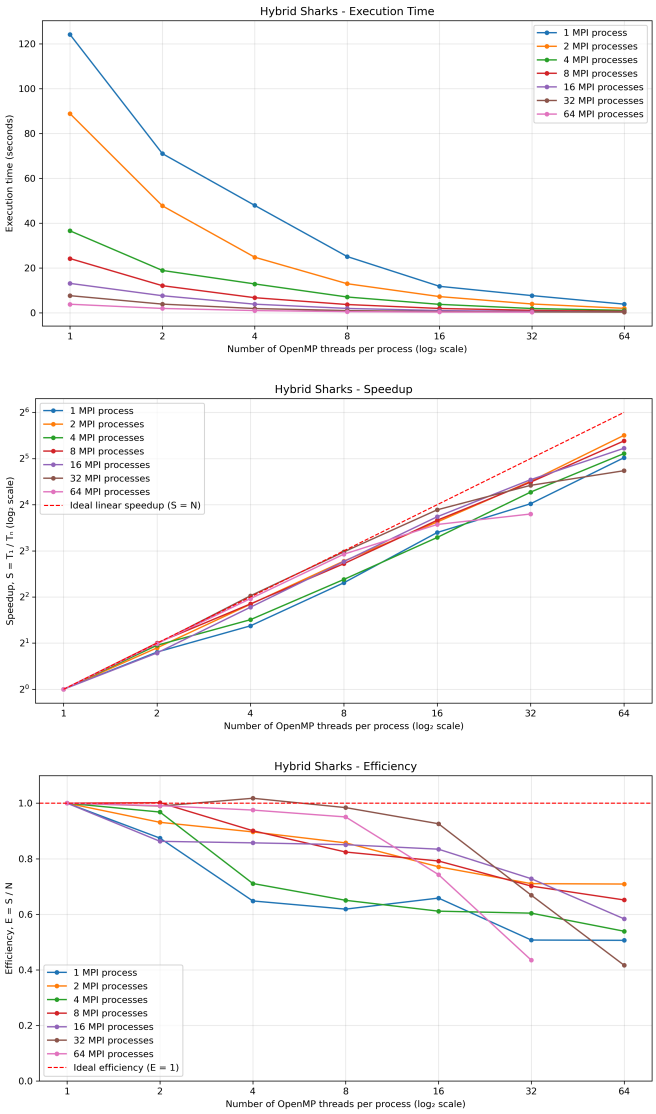


Fig. 7. Hybrid MPI+OpenMP shark-level implementation: execution time, speedup, and efficiency. Each line corresponds to a fixed number of MPI processes; the x-axis is the number of OpenMP threads per process.

For a fixed number of MPI processes, adding OpenMP threads reduces execution time near-linearly up to roughly 16–32 threads per process, after which gains diminish. With p=8 processes and t=64 threads (512 total cores), execution time drops to 0.54 s, giving a speedup of approximately 230× at 45% efficiency. The best raw result is p=64, t=32 (2048 total cores) at 0.277 s, which is 450× faster than the serial baseline of 124.6 s. That corresponds to a parallel efficiency of 22%, which reflects the thin per-rank population (roughly 16 sharks per MPI process) at that configuration.

Comparing hybrid to pure-MPI at the same total worker count $P \times T$ reveals a consistent advantage for the hybrid configuration at high total counts. Eight MPI processes with 8 threads each (64 total workers) complete in 3.71 s (speedup 33.4×, efficiency 52%), while pure MPI at 64 processes completes in 2.91 s (speedup 42.7×, efficiency 67%). Pure MPI wins at 64 total workers because the inter-process reduction is cheap for a small reduction vector. However, as the total

worker count grows further, the hybrid approach scales better: with p=16 and t=64 (1024 total workers) the time is 0.35 s, a regime where pure MPI at 1024 processes would face severe load imbalance since each rank would hold fewer than one shark.

Efficiency drops below 30% at very high combined counts (p=32, t=32 or p=64, t=16) because the local population per rank falls to 1–2 sharks and any imbalance in per-shark computation time is magnified. The practical operating point for this problem size is around p=8–16 processes and t=8–32 threads, yielding speedups of 50–230× with efficiencies of 30–52%.

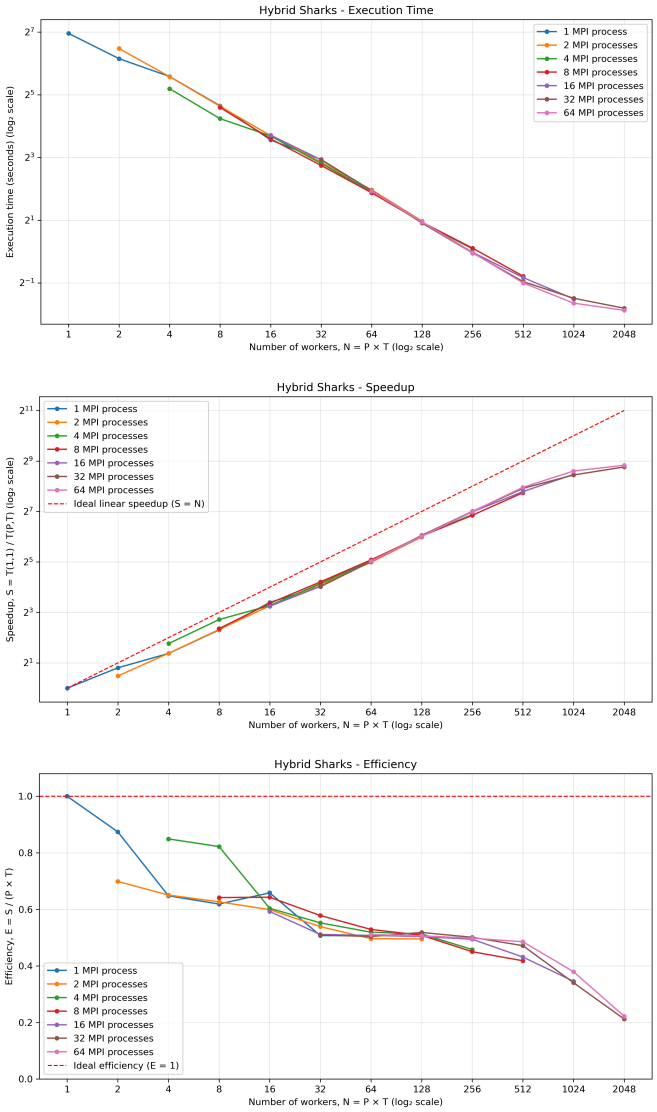


Fig. 8. Hybrid MPI+OpenMP shark-level implementation, global view: execution time, speedup, and efficiency measured against the serial baseline $T(1,1)$. Each line corresponds to a fixed number of MPI processes; the x-axis is the total number of workers $P \times T$ on a log2 scale.

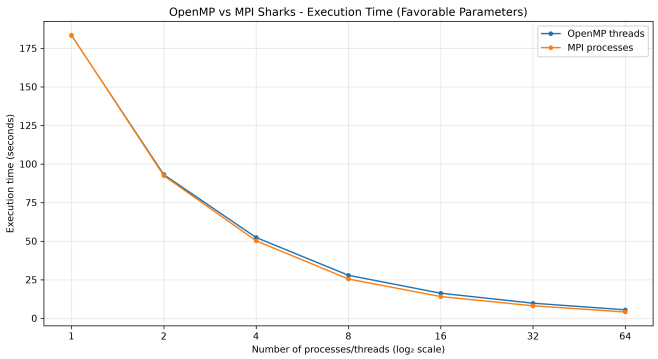
The plots in Fig. 7 normalize each curve to its own single-thread time, which isolates the effect of adding threads to a fixed process count but hides the comparison across different process counts. Fig. 8 instead normalizes every configuration to the true serial baseline `T(1,1)` and places the total worker count $P \times T$ on a common log2 axis, so that all hybrid configurations can be read against one another and against serial execution directly.

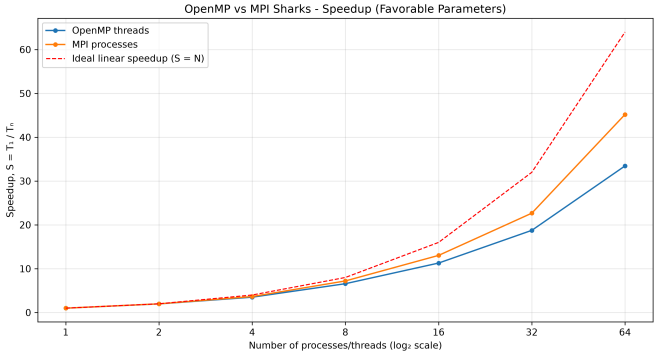
The efficiency plot is the clearest result: the curves for every process count collapse onto one envelope, falling from about 60% at low worker counts to about 22% at 2048 workers. Efficiency depends on the total worker count $P \times T$, not on how it is split between MPI processes and OpenMP threads. Trading processes for threads at a fixed total core count does not improve efficiency; it only changes whether that core count can be reached at all.

This reframes the pure-MPI versus hybrid comparison. Hybrid is not more efficient than pure MPI at equal worker count; it wins by reaching worker counts (1024, 2048) that pure MPI cannot, since there pure MPI would assign fewer than one shark per rank. The time curve confirms this: all splits fall on one time-versus-workers trajectory that bottoms out below one second, while speedup climbs to the 450× peak at 2048 workers. In short, efficiency stays at or above 45% while the total worker count is at most about 512, beyond which the per-rank population becomes too thin regardless of the split.

E. *Favorable configuration (shark-level)*

The fixed configuration starves the shark axis at high worker counts. The favorable shark configuration (`np` = 8192, see Table III) instead keeps every core supplied with sharks: 128 sharks per worker for the pure variants at 64 workers, and roughly eight sharks per core for the hybrid run at 1024 cores. Fig. 9 shows the pure MPI and OpenMP variants, and Fig. 10 the hybrid variant, each measured against the matching serial baseline.





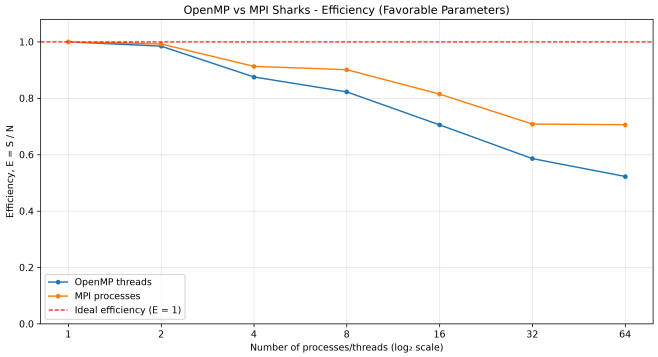
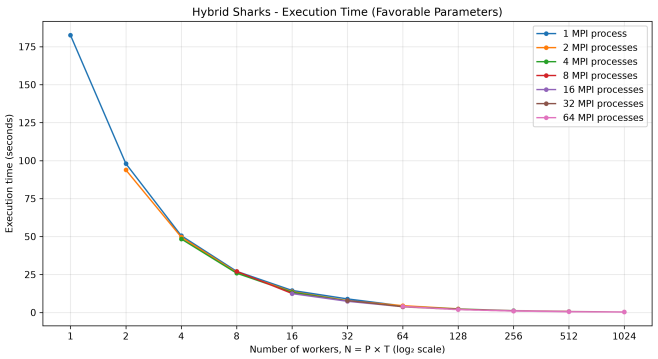
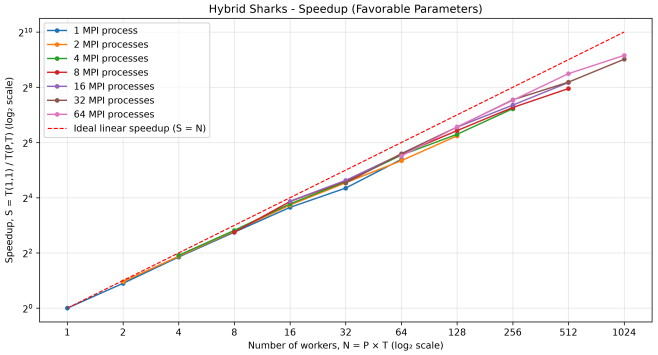


Fig. 9. Favorable shark configuration (np = 8192): execution time, speedup, and efficiency for MPI (processes) and OpenMP (threads).





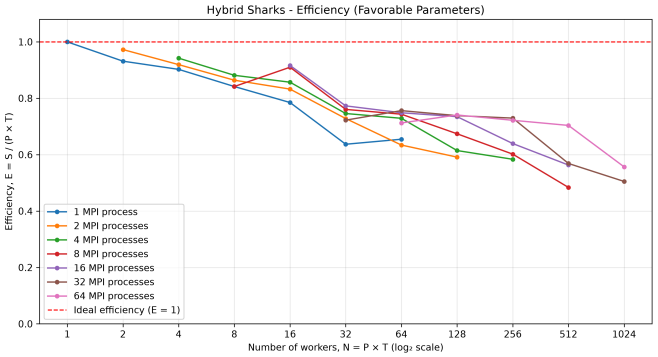


Fig. 10. Favorable shark configuration (np = 8192), hybrid MPI+OpenMP: execution time, speedup, and efficiency against the serial baseline.

The effect is exactly the one predicted by the starvation analysis. With a large population the efficiency no longer collapses at high worker counts: the pure variants stay strongly scalable across the full process and thread range, and the hybrid run sustains a far higher efficiency at 1024 cores than the fixed configuration did, where efficiency had fallen to about 22%. This confirms that the poor efficiency of the fixed hybrid at its peak was an artifact of insufficient population per core, not a limit of the decomposition. The decomposition strategy is unchanged; only the workload is sized to the machine.

F. *Favorable configuration (dimension-level)*

The favorable dimension configuration (nd = 10^8 with only 5 sharks and 5 stages, see Table III) makes the dimension axis dominant. Fig. 11 shows the OpenMP variant (the y-axis is log2 to make the small spread legible). Even with a dimension count far above the worker count, scaling plateaus early: execution time falls from about 111 s at one thread to

roughly 48 s at 8 threads, then flattens to about 42 s at 64 threads, a ceiling near 2.6×. With `M` = 0 there is no rotation search and the stage count is tiny, so synchronization is not the bottleneck; the limit is memory bandwidth. The position vector alone is about 800 MB, and every stage streams it through the per-dimension update, so once a few threads saturate the socket's bandwidth additional threads add little. Dimension-level decomposition is therefore bandwidth-bound rather than compute-bound, even at extreme `nd`.

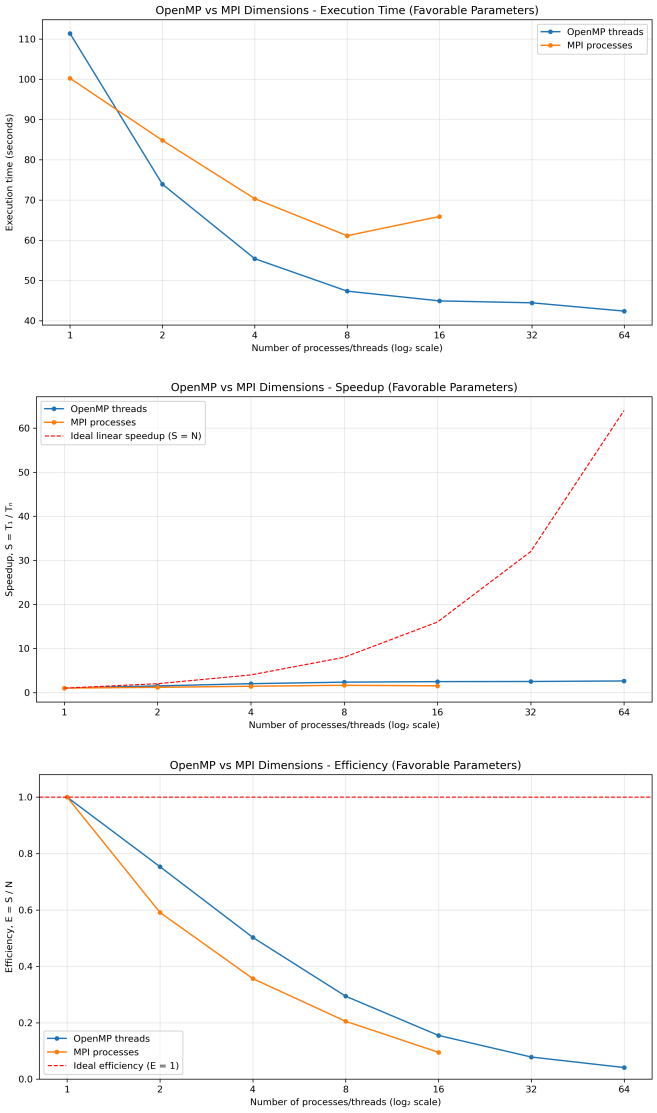


Fig. 11. Favorable dimension configuration (`nd` = 10^8): execution time (log2 y-axis), speedup, and efficiency for the OpenMP variant.

G. *Favorable configuration (rotation-level)*

The favorable rotation configuration (`M` = 20000 with 16 sharks, see Table III) makes the rotation axis dominant, with far more probes than workers. Fig. 12 shows both pure variants. The MPI variant scales strongly, reaching about 40× at 64 processes (roughly 63% efficiency): each rank evaluates a large independent slice of the 20000 probes, and the single per-stage `MAXLOC` reduction is cheap relative to that work. The OpenMP variant is weaker, reaching about 8× at 64 threads (roughly 13% efficiency) and flattening beyond 16 threads, limited by the per-stage barriers of its parallel region and the serial speed and position update that precedes the rotation loop. Either way, the result confirms that the poor rotation scaling under the fixed configuration was a granularity artifact of `M` = 50, not an algorithmic limit: given enough probes, rotation-level parallelism scales, most effectively under MPI.

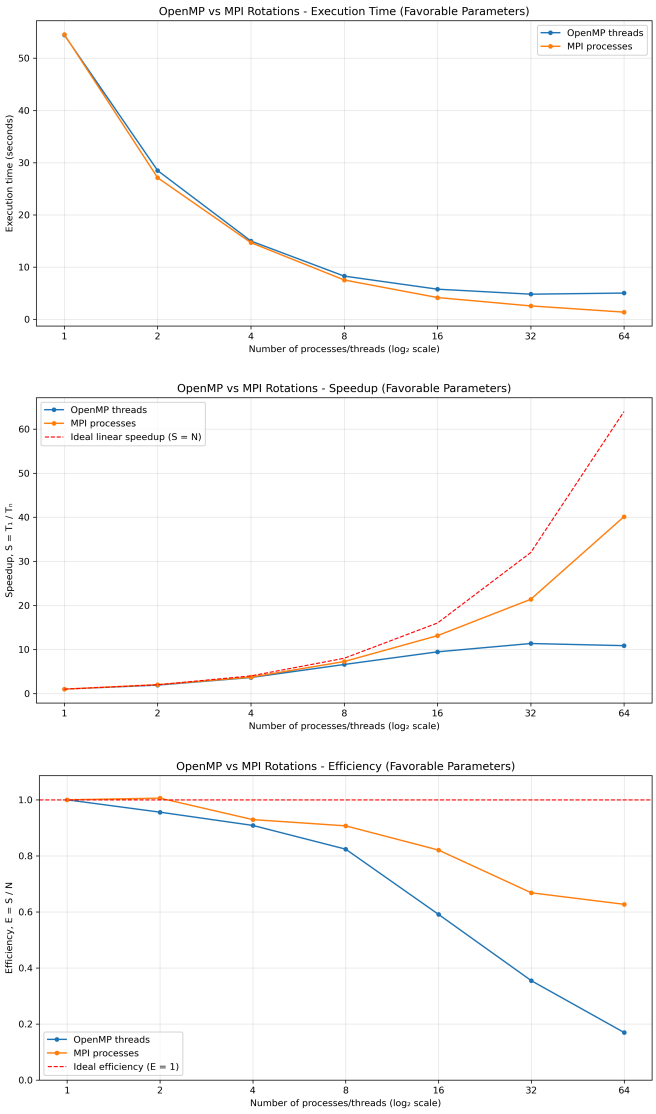
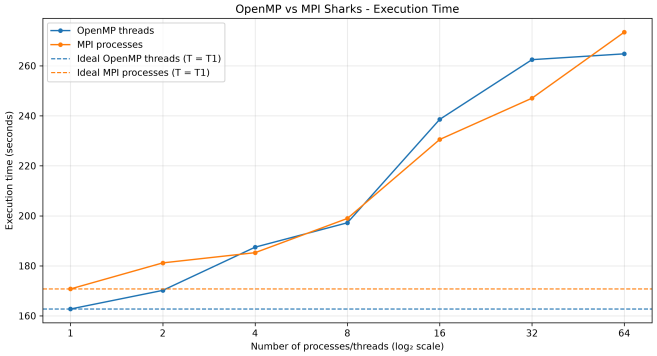
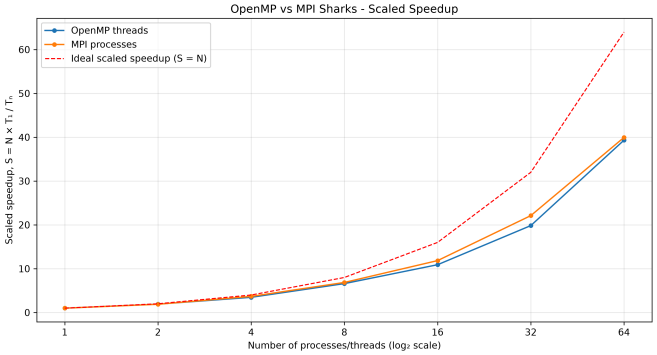


Fig. 12. Favorable rotation configuration (`M` = 20000): execution time, speedup, and efficiency for MPI (processes) and OpenMP (threads).

H. *Weak scaling (shark-level)*

All results so far are strong scaling: a fixed problem solved with more workers. We also evaluate weak scaling, where the population grows in proportion to the worker count so that the per-worker load stays constant. Fig. 13 reports the pure MPI and OpenMP variants and Fig. 14 the hybrid variant; ideal weak scaling appears as flat execution time and efficiency near one.





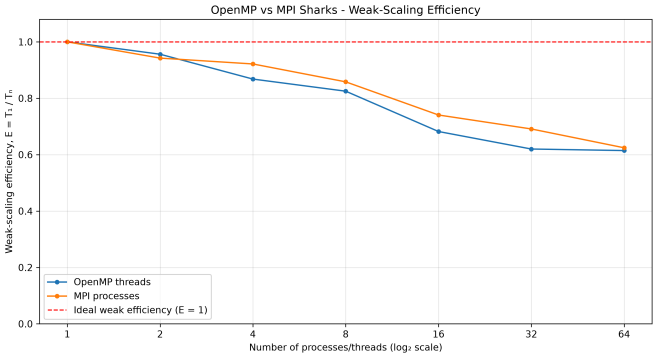
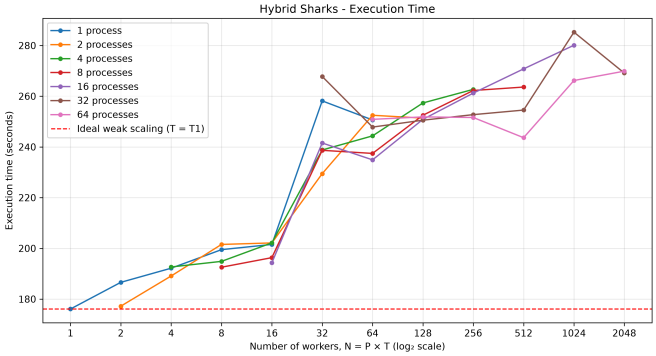
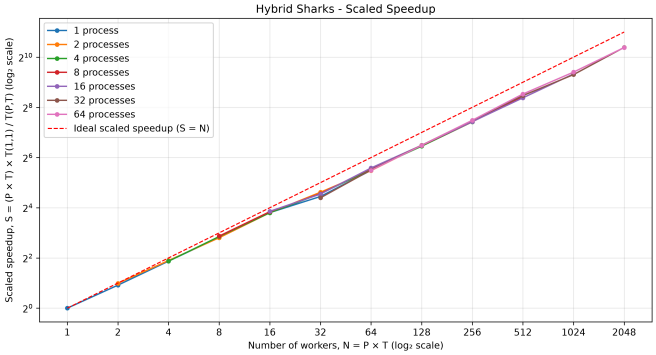


Fig. 13. Weak scaling, shark-level: execution time, speedup, and efficiency for MPI (processes) and OpenMP (threads) as the population grows with the worker count.





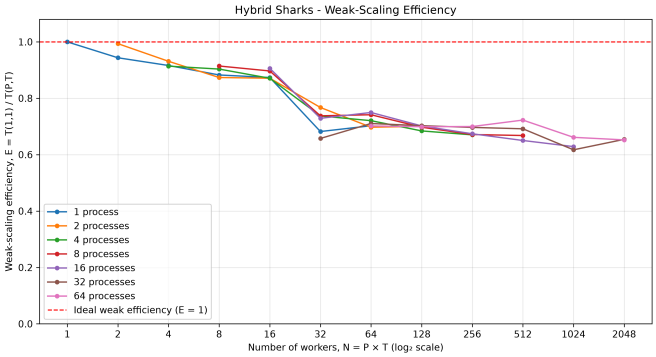


Fig. 14. Weak scaling, hybrid MPI+OpenMP shark-level: execution time, speedup, and efficiency as the population grows with the worker count.

Because shark-level decomposition coordinates only through one final reduction, the per-worker work is essentially independent of the worker count, so execution time stays nearly flat as the population and workers grow together. The weak-scaling efficiency remains high and degrades only slowly, driven by the growing cost of the final reduction rather than by the per-shark computation. This is the complementary view to the strong-scaling results: shark-level decomposition is both strongly and weakly scalable.

I. Weak scaling (dimension-level)

For the dimension axis the per-worker load is held constant by growing the problem dimensionality `nd` in proportion to the worker count (100000 dimensions per worker, with `np` = 5, k_max = 20 and no rotation). Fig. 15 reports the pure MPI and OpenMP variants.

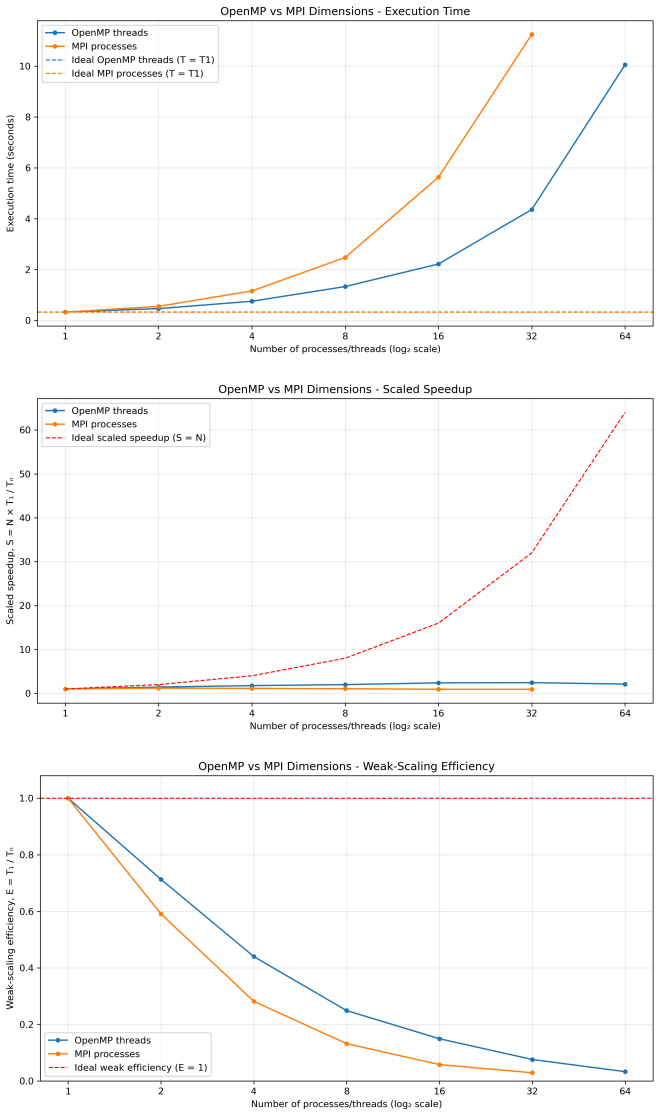


Fig. 15. Weak scaling, dimension-level: execution time, speedup, and efficiency for MPI (processes) and OpenMP (threads) as `nd` grows with the worker count.

Dimension-level decomposition is not weakly scalable. Execution time rises steeply with the worker count instead of staying flat: the MPI variant grows from 0.33 s to 11.2 s across 1 to 32 processes and the OpenMP variant from 0.33 s to 10.0 s across 1 to 64 threads, so weak-scaling efficiency falls to a few percent. The cause is the one already seen under strong scaling: every stage reassembles the full position vector, so communication and memory-bandwidth demand grow with the worker count even though each worker owns a constant slice of dimensions. This confirms that the strategy is bandwidth-bound and viable only with a much heavier per-dimension kernel.

J. *Weak scaling (rotation-level)*

For the rotation axis the per-worker load is held constant by growing the number of rotational probes `M` with the worker count (300 probes per worker, with `np` = 16, nd = 200 and k_max = 100). Fig. 16 reports the pure MPI and OpenMP variants.

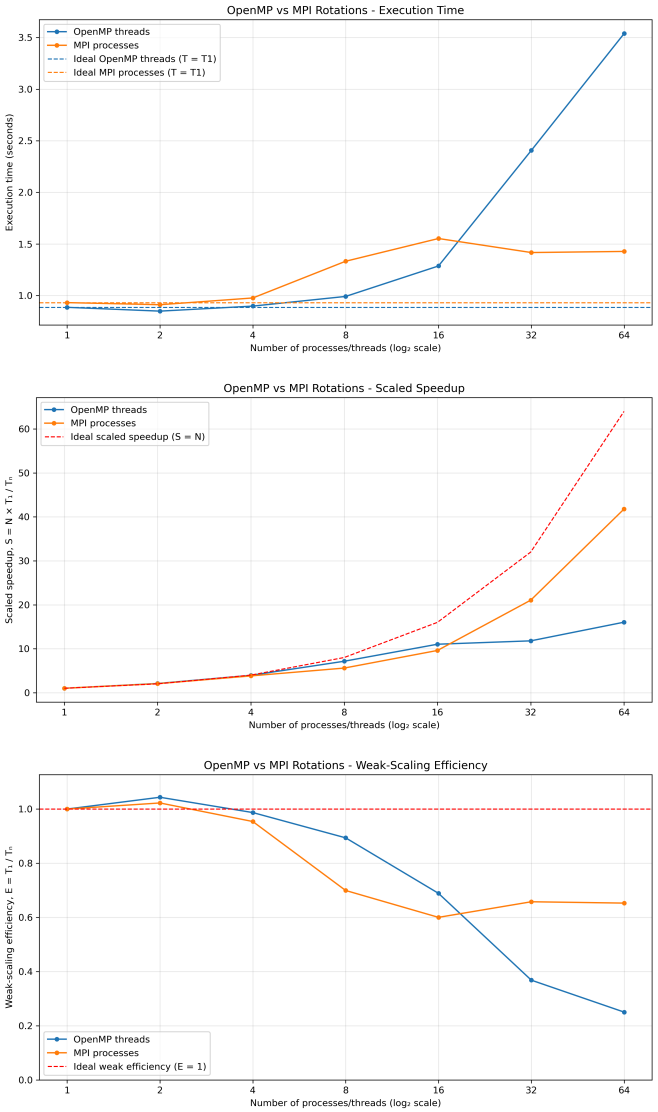


Fig. 16. Weak scaling, rotation-level: execution time, speedup, and efficiency for MPI (processes) and OpenMP (threads) as `M` grows with the worker count.

Rotation-level decomposition weak-scales well, especially under MPI. The MPI variant stays nearly flat, from 0.93 s at one process to 1.43 s at 64 (about 65% weak-scaling efficiency), because the only coordination is a two-value reduction whose cost is independent of `M`. The OpenMP variant holds high efficiency up to 16 threads (about 69%) and then degrades, reaching 3.54 s at 64 threads: as more threads share one node, the growing memory traffic of the rotational probes and the cost of the per-stage reduction across threads erode the constant per-thread work. The MPI result complements the favorable-rotation strong-scaling finding: when the per-unit workload is sufficient, rotation-level parallelism is both strongly and weakly scalable.

K. Playing with PBS

All experiments were executed on the HPC cluster using the PBS scheduler in `excl` mode to prevent resource sharing with other jobs and ensure reproducible wall-clock measurements. Jobs were submitted via the scripting infrastructure described in Listing 6 and Listing 7.

Placement policy. The PBS `place` directive controls how the allocated chunks (nodes) are mapped to the physical topology. All runs used `scatter`, which allocates each chunk on a distinct physical node. This maximizes aggregate memory bandwidth and gives each MPI rank exclusive access to a full node's cores and memory, yielding predictable per-rank core counts. For the hybrid tests `scatter` is used as well, since intra-node placement is already handled by the `--map-by ppr:N:node:PE=T` option and scattering nodes prevents inadvertent sharing. The alternative `pack` policy, which packs chunks onto shared nodes to cut inter-node latency, was not needed for these workloads.

Walltime and queue. All jobs were submitted to the `shortCPUQ` queue. Sharks-level tests used a 5-minute walltime; rotation and dimension tests used 15 minutes because single-process runs are longer.

V. Conclusion

This paper studied seven parallel variants of the Shark Smell Optimization algorithm: three decomposition strategies (shark-level, dimension-level, rotation-level) under both MPI and OpenMP, plus a hybrid MPI+OpenMP implementation at the shark level.

The central finding is that the decomposition strategy matters far more than the programming model. Shark-level decomposition is highly parallel and scales almost linearly with worker count in both MPI and OpenMP, reaching 43 $\times$ at 64 MPI processes and 50$\times$ at 64 OpenMP threads on the Rastrigin benchmark. Dimension-level and rotation-level decompositions both fail to scale under the standard parameter set because their work granularity (a single gradient evaluation or a single rotation probe) is too small to amortize synchronization and communication costs. With a larger rotation count, rotation-level MPI recovers and reaches 20$\times$ at 64 processes, confirming that these strategies are viable only when the per-unit workload is sufficient.

The hybrid MPI+OpenMP shark-level variant combines the two scalable axes and achieves the shortest wall-clock time: 0.277 s at p=64, t=32 (2048 total cores), a 450$\times$ reduction over the serial baseline at 22% efficiency. The practical operating point for the standard problem size is p=8–16 processes and t=8–32 threads, which yields 50$\times$–230$\times$ speedup at 30–52% efficiency with far fewer resources.

These efficiency figures are bounded by the fixed configuration itself: at high worker counts the population is too small to keep every core busy. Re-running the shark variants with a favorable population (`np` = 8192) confirms this. The pure MPI variant reaches 45$\times$ at 64 processes (71% efficiency) and the hybrid variant reaches roughly 570$\times$ at 1024 cores (about 56% efficiency), more than double the efficiency of the fixed hybrid at its peak and with half the cores. The same favorable workload is also weakly scalable: execution time stays nearly flat as the population grows with the worker count, since shark-level decomposition coordinates only through a single final reduction. The decomposition is therefore both strongly and weakly scalable when the workload is sized to the machine.

Two limitations apply broadly. First, population-level load balance relies on uniform per-shark cost; if the objective function evaluation becomes problem-dependent and variable, work stealing or dynamic scheduling would be needed. Second, none of the variants exploit GPU acceleration, which would be particularly natural for the embarrassingly parallel rotation search and could extend the speedup by another order of magnitude.

VI. Use of AI Tools

AI assistance was used in two supporting roles. First, to help develop the Python plotting scripts (`benchmarks/plot_results.py`) that turn the raw benchmark logs into the execution-time, speedup, and efficiency figures shown throughout this paper. Second, to revise some passages of the report for clarity and consistency, including parts of the parallel-design and performance sections. All algorithmic design, parallel implementations, experiments, and the resulting data and conclusions are our own; AI-generated text and code were reviewed and validated by the authors.

References

[1] A. Campelo, "EC Bestiary: A bestiary of evolutionary, swarm and other metaphor-based algorithms," *Zenovo*, [Online]. Available: https://fcampelo.github.io/EC-Bestiary/

[2] O. Abedinia, N. Amjady, and A. Ghasemi, "A new metaheuristic algorithm based on shark smell optimization," *Complexity*, vol. 21, no. 5, pp. 97–116, 2016, doi: https://doi.org/10.1002/cplx.21634.

[3] G. V. Pietro Cipriani Filippo De Grandi, "Parallel Shark Smell Optimization (SSO)." [Online]. Available: https://github.com/Degra02/parallel-sso

[4] Wikipedia, "Rastrigin function," [Online]. Available: https://en.wikipedia.org/wiki/Rastrigin_function